UNIVERSITY of WASHINGTON

UW Math AI Lab

Spring 2026 · Provable Computation

VERIFIED AND EXECUTABLE LINEAR ALGEBRA

DenseMatrix: a computationally efficient matrix representation in Lean.

Row-major array representation for matrix computations, with a ring isomorphism to the proof-friendly mathlib Matrix.

TEAM

Annis Wu · Joseph Qian · Junye Ji · Veer Shukla

Mentor: Dhruv Bhatia

SCAN FOR ARTIFACT

Why DenseMatrix

- **The Problem:** Current mathlib representation is function-backed. This is great for theory, but terrible for computation speeds.
- **The Solution:** We defined a new matrix representation using a row-major 1D array to improve runtime for linear algebra algorithms.
- **The Bridge:** We proved a ring isomorphism exists between DenseMatrix and mathlib Matrix. Algorithms are written in DenseMatrix for speed, but proven using mathlib Matrix (transporting theory via the isomorphism).
- **Next Steps:** Porting our existing algorithms to this new, computationally-efficient data type.

Tail Recursion

Lean uses a functional but in-place paradigm: it optimizes memory by updating values in-place *only* if there is a single reference to that value. If the reference count exceeds 1, array mutation requires O(n) time, significantly slowing down algorithms.

While the caller ensures the initial matrix has a reference count of 1, we rely on *tail recursion* to ensure that count doesn't increase during recursive multiplication.

Functional languages like Lean use "tail-call elimination". If a function call is in "tail position," the compiler replaces the current stack frame instead of pushing a new one. This keeps the reference count constant and ensures efficient array mutation.

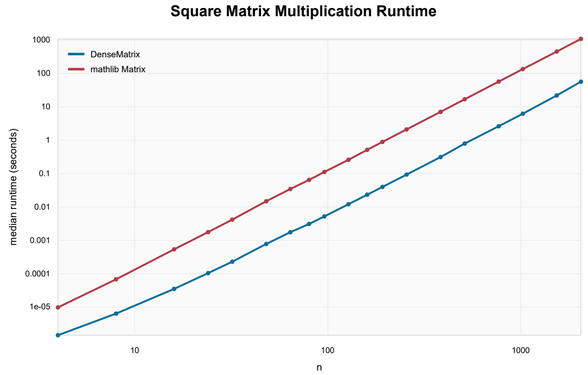
Definition: A function call is in tail position if the caller doesn't need to modify its returned value before returning. Notably:

- If a match-expression is in tail position, each branch is too.
- If an if-expression is in tail position, both branches are too.
- If a let-expression is in tail position, its body is too.

See Functional Programming in Lean Ch. 8.1 for more details.

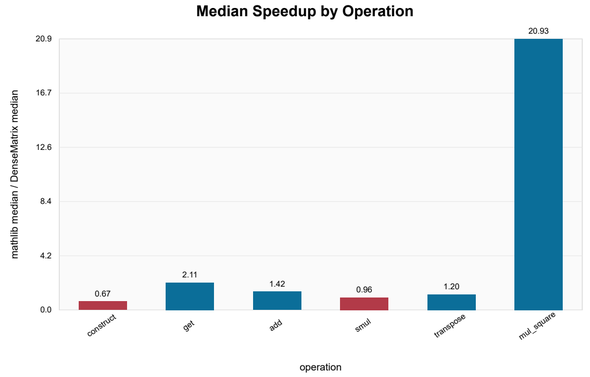
Multiplication Scaling

DenseMatrix vs mathlib Matrix multiplication median runtime



Representation Tradeoff

DenseMatrix vs mathlib Matrix median speedup



Benchmark Setup: 30 repeats, 5 warmups · Pinned CPU core · Sizes: 16 to 1024